

PYTHON & SQL INTERVIEW NOTES

Functions (defaults, *args, **kwargs, scope) · NULL handling with COALESCE ·
GROUP BY / HAVING · Subqueries & correlated subqueries

PYTHON

SQL

INTERVIEW PREP

• Python — Functions

Default arguments, *args / **kwargs, scope, and a practical discount-calculation problem.

1 Default Arguments

```
def add(a, b=10):  
    return a + b  
  
add(100) # 110  
add(100, 20) # 120
```

REMEMBER

If no value is provided, the default value is used.

2 *args — Variable Positional Arguments

Used when the function can receive any number of positional arguments.

```
def calculate_average(*args):  
    return sum(args) / len(args)  
  
calculate_average(10, 20, 30)  
# 20.0
```

REMEMBER

args is a tuple.

3 ****kwargs** — Variable Keyword Arguments

Used when the function can receive any number of keyword arguments.

```
def employee_info(**details):  
    print(details["salary"])  
  
employee_info(  
    name="Alice",  
    salary=100000,  
    department="IT"  
)
```

REMEMBER

kwargs is a dictionary.

IMPORTANT INTERVIEW DIFFERENCE

*args → tuple → positional arguments | **kwargs → dict → keyword arguments

4 Combining Arguments

```
def employee(name, *skills, **details):  
    print(name)  
    print(skills)  
    print(details["salary"])
```

Example:

```
employee(  
    "Alice",  
    "Python", "SQL", "Pandas",  
    salary=100000,  
    department="IT"  
)
```

5 Scope

```
x = 10

def test():
    x = 20
    print(x)

test()
print(x)
```

Output:

```
20
10
```

$x = 20 \rightarrow \text{local}$ $x = 10 \rightarrow \text{global}$

To modify the global variable:

```
x = 10

def change():
    global x
    x = 20
```

6 Practical Function Problem

Discount calculation:

```
def calculate_total(*prices, discount=0):
    total = sum(prices)
    discount_amount = total * (discount / 100)
    return total - discount_amount

calculate_total(100, 200, 300, discount=10)
# 540.0
```

• SQL — NULL Handling, Aggregates & Subqueries

Practiced on the Employee_2 table: COALESCE, GROUP BY / HAVING, and correlated subqueries.

1 COALESCE() — Replacing NULL

Used to replace NULL with another value.

```
SELECT name,  
       COALESCE(salary, 0) AS salary  
FROM Employee_2;
```

REMEMBER

COALESCE(value, replacement)

2 NULL Filtering

```
SELECT name, bonus  
FROM Employee_2  
WHERE bonus IS NULL;
```

IMPORTANT

Don't use: `WHERE bonus = NULL` — use: `WHERE bonus IS NULL`

3 COALESCE() in Calculations

```
SELECT name,  
       COALESCE(salary, 0) + COALESCE(bonus, 0)  
       AS total_compensation  
FROM Employee_2;
```

This prevents NULL from making the entire calculation NULL.

4 AVG() + GROUP BY

```
SELECT departmentId,  
       AVG(salary) AS average_salary  
FROM Employee_2  
GROUP BY departmentId;
```

IMPORTANT INTERVIEW POINT

AVG(salary) automatically ignores NULL salaries. Don't unnecessarily do `AVG(COALESCE(salary, 0))` — that treats missing salaries as 0 and changes the average.

5 HAVING

Find departments whose average salary is above 75,000:

```
SELECT departmentId,  
       AVG(salary) AS average_salary  
FROM Employee_2  
GROUP BY departmentId  
HAVING average_salary > 75000;
```

REMEMBER

WHERE → filters rows HAVING → filters groups

6 Highest Average-Salary Department

```
SELECT departmentId,  
       AVG(salary) AS average_salary  
FROM Employee_2  
GROUP BY departmentId  
ORDER BY average_salary DESC  
LIMIT 1;
```

GROUP BY
↓
AVG()
↓
ORDER BY DESC
↓
LIMIT 1

• Subqueries — Important Interview Pattern

Scalar, correlated, and MAX-based subquery patterns for comparing against a calculated value.

7 Employees Above Company Average

```
SELECT e.name,  
       e.salary  
FROM Employee_2 e  
WHERE e.salary > (  
    SELECT AVG(e2.salary)  
    FROM Employee_2 e2  
);
```

Inner query → company average
↓
Compare every employee's salary
↓
Keep salary > average

8 Employees Above Their Department Average

```
SELECT e.name,  
       e.salary,  
       e.departmentId  
FROM Employee_2 e  
WHERE e.salary > (  
    SELECT AVG(e2.salary)  
    FROM Employee_2 e2  
    WHERE e2.departmentId = e.departmentId  
);
```

This is a **correlated subquery**.

KEY LINE

WHERE e2.departmentId = e.departmentId — it makes the inner query calculate the average for the current employee's department.

9 Highest-Paid Employee in Each Department

```
SELECT e.name,  
       e.salary,  
       e.departmentId  
FROM Employee_2 e  
WHERE e.salary = (  
    SELECT MAX(e2.salary)  
    FROM Employee_2 e2  
    WHERE e2.departmentId = e.departmentId  
);
```

IMPORTANT

It can return multiple employees if two employees have the same highest salary.

10 Second-Highest Salary

```
SELECT MAX(salary) AS second_highest_salary  
FROM Employee_2  
WHERE salary < (  
    SELECT MAX(salary)  
    FROM Employee_2  
);
```

Dry-run pattern:

1. Find highest salary
↓
2. Remove highest salary
↓
3. Find MAX() of remaining salaries
↓
4. Second-highest salary

★ Most Important Patterns From Today

Quick-recall summary for last-minute revision.

Python

```
default argument
↓
*args → tuple
↓
**kwargs → dictionary
↓
local / global scope
↓
practical function combinations
```

SQL

```
NULL → COALESCE()
NULL check → IS NULL
GROUP BY → aggregate per group
HAVING → filter groups
Subquery → compare against calculated value
Correlated subquery → compare within each department
MAX + subquery → highest / second-highest patterns
```